Dunder Mifflin - Application and design specification

Name:	Amogh Doddagadduvalli Prasanna
Email:	amoghdp@umd.edu
UMD directory ID:	amoghdp
Application name:	Dunder Mifflin

Table of Contents

List of Figures ---------- 3

System Architecture ---------- 5

Application Software Components ---------- 6

Application Data Components ---------- 10

Security Architecture ---------- 13

Security Components ---------- 18

Logging Design ---------- 21

References ---------- 26

List of Figures

- **Figure 1**: System Architecture Relationships ---------- 5
- **Figure 2**: Dunder Mifflin Software Components ---------- 6

- **Figure 3**: Classes ---------- 6
- **Figure 4**: Services (Business Logic) ---------- 7
- **Figure 5**: Dunder Mifflin UI Components ---------- 8
- **Figure 6**: Dunder Mifflin Login Page ---------- 8
- **Figure 7**: Dunder Mifflin Home Page ---------- 9
- **Figure 8**: Database Table Structure ---------- 11

1. Application Overview

- **High-Level Description:**

The **Dunder Mifflin Inventory System** is a web-based application designed to support product catalog management, real-time stock tracking, and role-based user operations (Sales Associate, Warehouse Employee, Manager). It complements the existing public-facing site by introducing authenticated inventory management capabilities.

- **Technology Stack:**
    - **Frontend:** HTML/CSS/JS (static, extendable to React + Tailwind).
    - **Backend:** Go Fiber v2 (RESTful API).
    - **Database:** PostgreSQL with migrations and triggers.
    - **Auth:** JWT-based authentication, role-based authorization.

- **Deployment:** Docker Compose (API + DB).

- **Architecture Approach**

A **3-tier layered architecture** (Presentation, Application, Data) with separation of concerns:

- Presentation Tier: UI for employees/managers.
- Application Tier: REST API (Go Fiber) for business logic and rules.
- Data Tier: PostgreSQL for persistence with an immutable inventory ledger.

2. System Architecture

- **Connections:**
  - Web UI interacts with REST API (JSON).
  - REST API communicates with PostgreSQL via a Data Access Layer.
  - Authentication middleware validates JWTs before processing requests.

- **Data Flow:**
  - User → Web UI → API request
  - API → Business Logic → Data Access
  - Data Access → DB (read/write)
  - DB response → Data Access → Business Logic → API → Web UI

- **Design Flow:**

[Sales Associate / Warehouse Manager /Employee] --> [Web UI] --> [Service API] --> [Business Logic] --> [Data Access Layer] --> [PostgreSQL DB]

System Architecture Design:

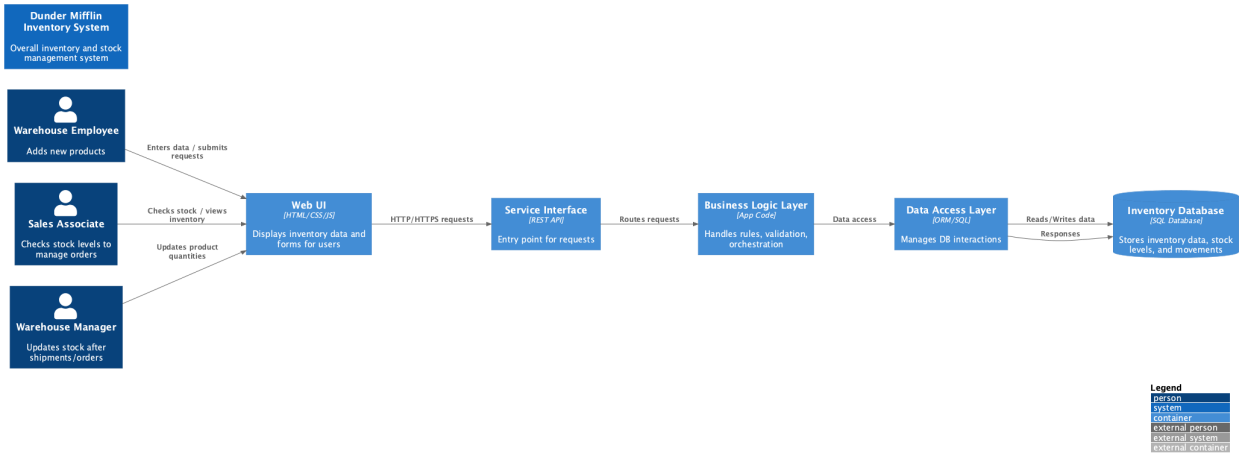


Figure 1: Architecture relationship

3. Application Software Components

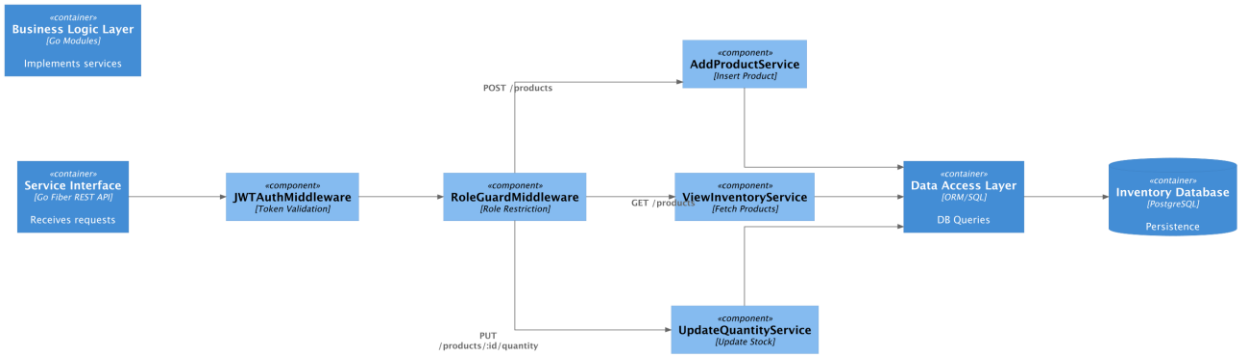


Figure 2: Dunder Mifflin Software Components

- **Models:**
  - Product (id, sku, name, description, unit, price, qty_on_hand, low_stock_threshold)
  - User (id, email, password_hash, role)
  - InventoryMove (id, product_id, kind, quantity, ref, actor_id, created_at)

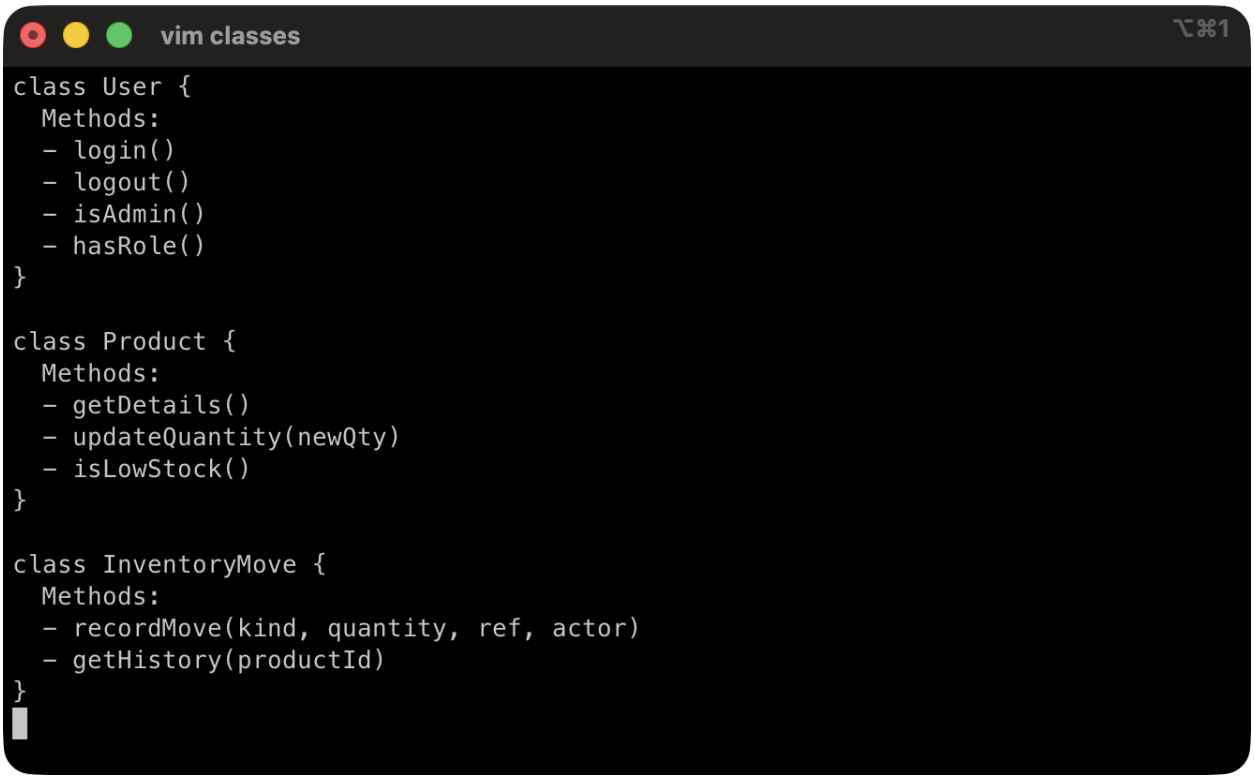


Figure 3: Classes

- **Business Logic:**
  - AddProductService → validates, inserts product.
  - ViewInventoryService → fetches inventory, applies filters.
  - UpdateQuantityService → updates quantity, records move.

- **Services/Controllers:**
  - /products (list/create)
  - /products/:id/quantity (update quantity)
  - /inventory/:id/moves (ledger)

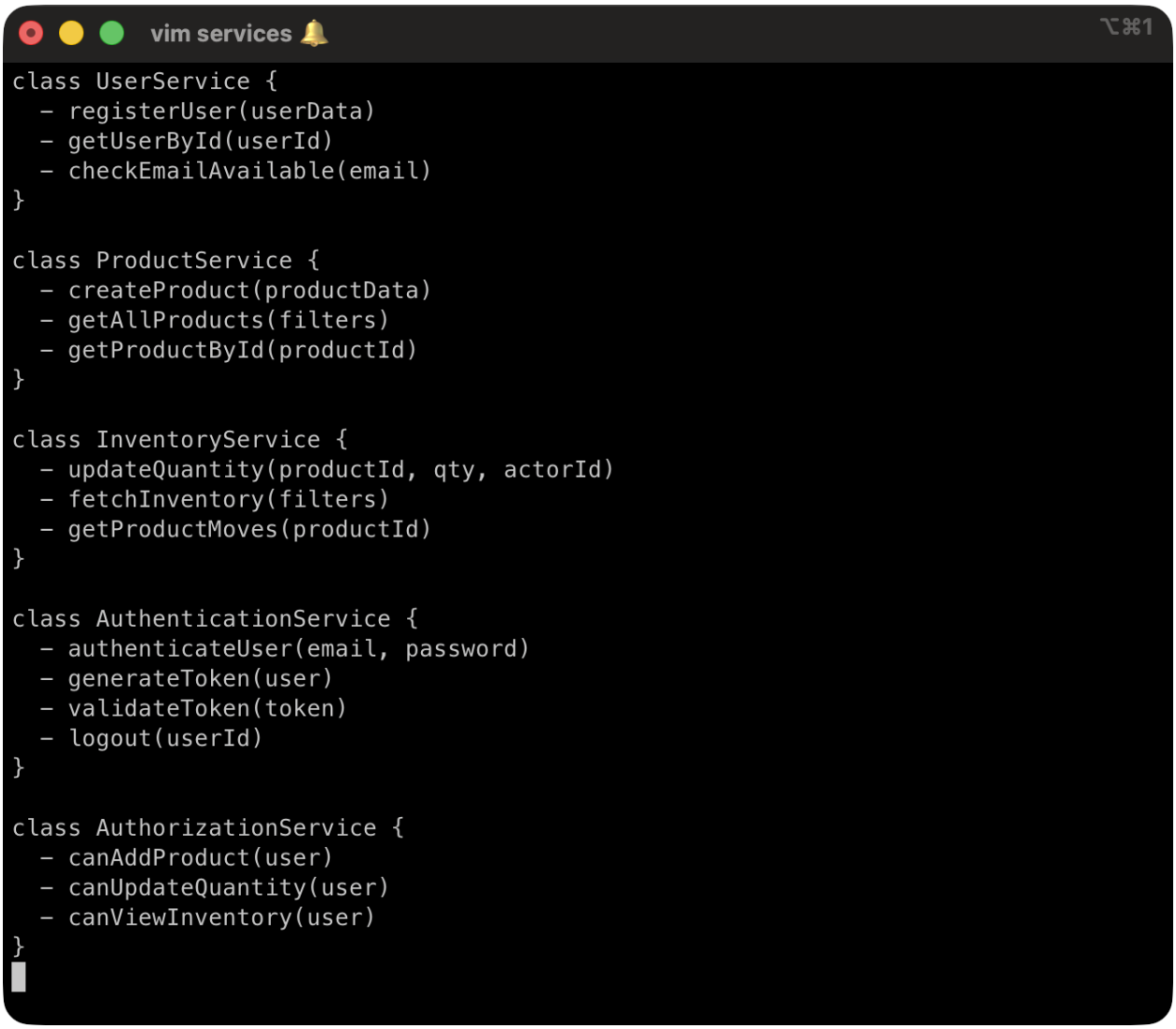


Figure 4: Services (Business Logic)

- **Middleware:**
  - JWTAuthMiddleware (validates token)
  - RoleGuardMiddleware (restricts routes by role).

4. Application UI Components

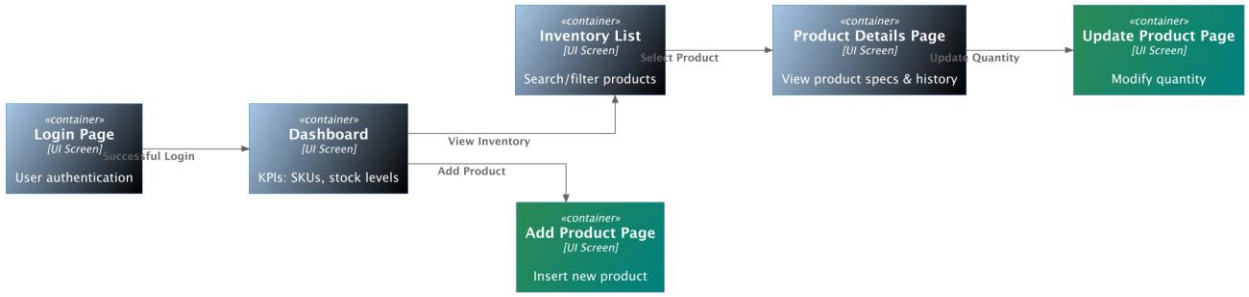


Figure 5: Dunder Mifflin UI Components

- **Pages/Screens:**
    - **Login Page** → user authentication.

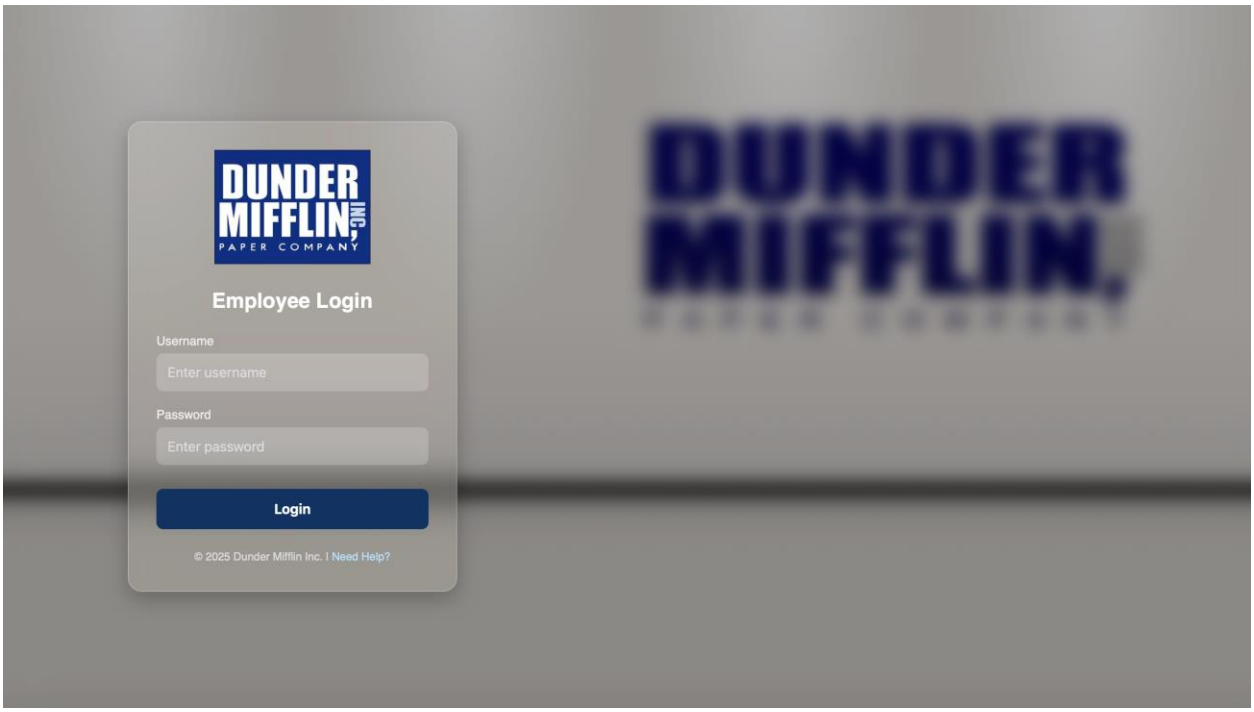


Figure 6: Dunder Mifflin Login Page

    - **Dashboard** → KPIs (total SKUs, in-stock, low-stock) -> Landing Page

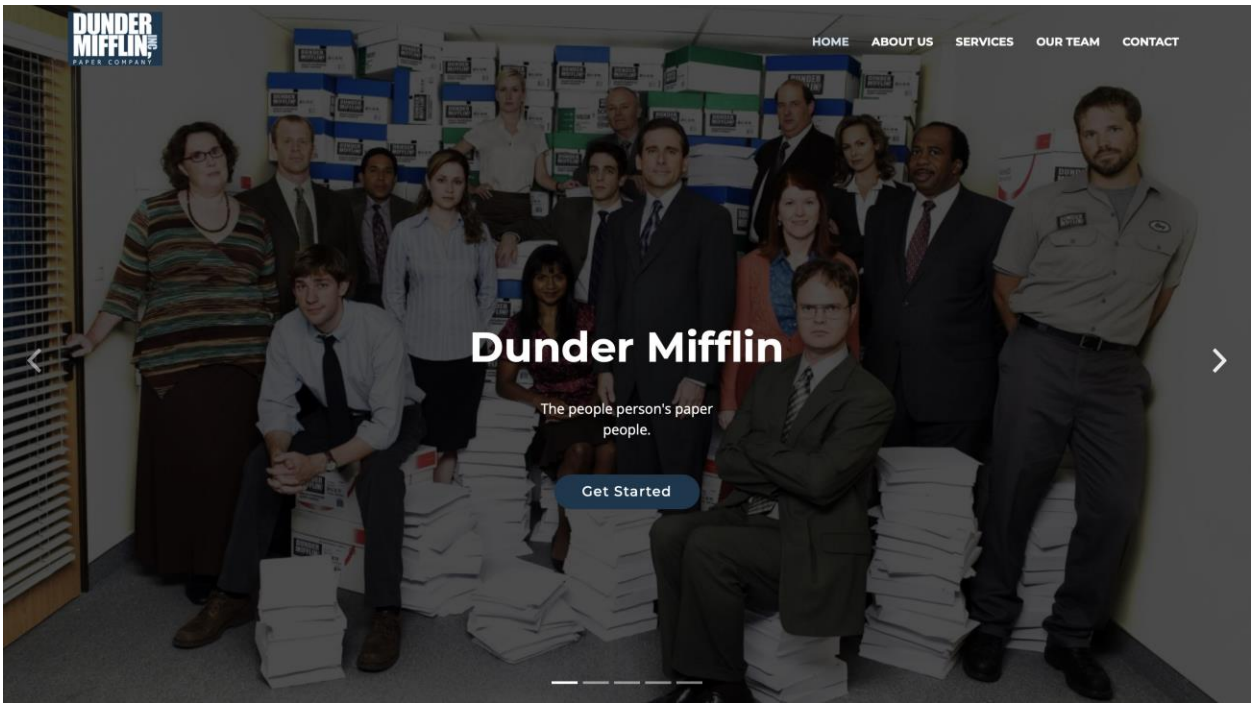


Figure 7: Dunder Mifflin Home Page

- **Inventory List** → search/filter products, quantity badges.
- **Add Product Page** → form to insert product.
- **Update Product Page** → modify quantity.
- **Product Details Page** → stock history + specifications.

- **Navigation:**
  - Dashboard → Inventory → Product Details → Update/Add.

5. Application Data Components

- **Database Tables:**
  - users(user_id, email, password_hash, role, created_at)
  - products(product_id, sku, name, description, unit, price_cents, qty_on_hand, low_stock_thr, created_at, updated_at)

- inventory_moves(move_id, product_id, kind, quantity, ref, actor_id, created_at)

- **Relationships:**
  - users (1) → inventory_moves (N)
  - products (1) → inventory_moves (N)

Database table structure:

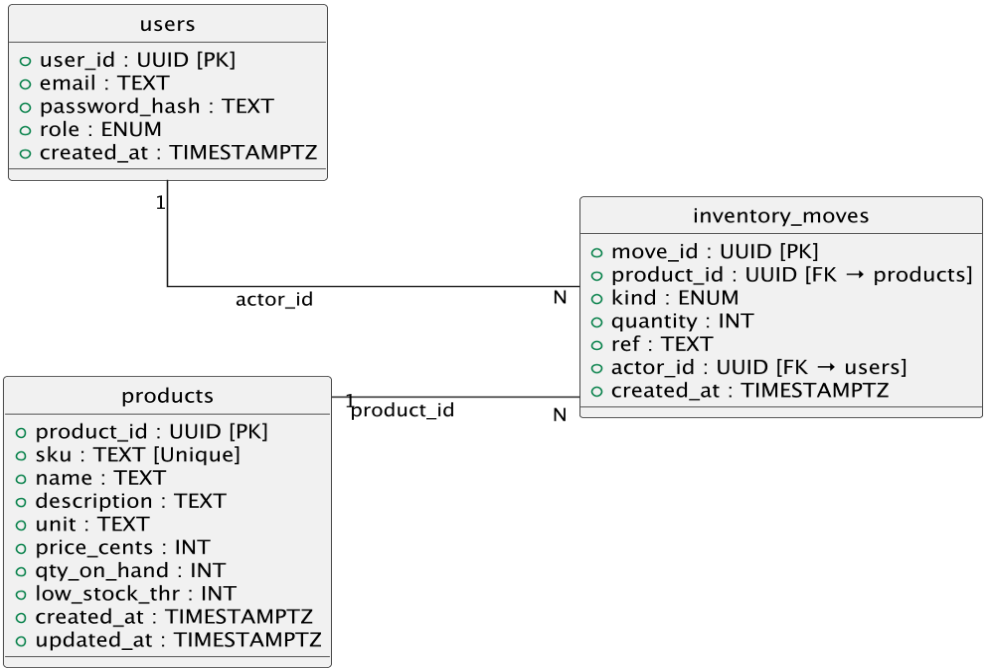


Figure 8: Database table structure

6. Test Cases

Test Case ID	Requirement ID	Description	Success Criteria
TC-001	FR-001	Add product with valid fields	Product exists in DB
TC-002	FR-001	Add duplicate SKU	API returns conflict error
TC-003	FR-002	Fetch inventory list	List matches DB
TC-004	FR-002	Filter low stock	Only products below threshold shown
TC-005	FR-003	Update product quantity	Quantity updated in DB
TC-006	FR-003	Submit invalid negative qty	API rejects with error

7. Traceability Matrix

Requirement ID	Use Case Ref	Design Component(s)	Test Case(s)
FR-001	UC-1 Add New Product	AddProductForm, ProductDAO	TC-001, TC-002

FR-002	UC-2 View Stock	InventoryPage, InventoryDAO	TC-003, TC-004
FR-003	UC-3 Update Quantity	UpdateForm, ProductDAO	TC-005, TC-006

8. Security Architecture

This section describes the comprehensive security architecture implemented to protect the Dunder Mifflin Inventory System from threats identified in the Requirements Specification, including misuse cases and STRIDE threat analysis.

8.1 Input Validation Architecture

8.1.1 Validation Layers

The system implements defense-in-depth with multiple validation layers:

- **Client-Side Validation:** JavaScript form validation for immediate user feedback (usability only, not security)
- **API Gateway Validation:** Fiber middleware validates content-type, payload size, and request format
- **Business Logic Validation:** Comprehensive whitelist validation in service layer
- **Database Constraints:** PostgreSQL constraints and triggers as final defense layer

8.1.2 Validation Rules Implementation

Field Type	Validation Method	Implementation
Product Name	Regex whitelist: [A-Za-z0-9 .-] Max 100 chars	Go validator package
Quantity	Integer type check, Range: 0-999999	strconv.Atoi() with bounds check
Email	RFC 5322 format validation	net/mail package
SQL Queries	Parameterized queries only	database/sql with $1, $2 placeholders

SQL Injection Prevention:

- All database queries use parameterized statements exclusively
- String concatenation for SQL is forbidden in codebase
- ORM/query builder validates and escapes all inputs
- Code review checklist includes SQL injection check

XSS Prevention:

- All dynamic content is HTML-encoded before rendering

- Content Security Policy (CSP) headers restrict script sources
- Input sanitization strips HTML/JavaScript tags from user input

8.2 Authentication and Authorization Architecture

8.2.1 Authentication Flow

Session-based authentication with secure cookie management:

- **Step 1:** User submits credentials via HTTPS POST to /api/v1/auth/login
- **Step 2:** System validates username exists in database
- **Step 3:** bcrypt.CompareHashAndPassword() verifies password
- **Step 4:** On success, create session with UUID and store in database
- **Step 5:** Set secure cookie (HTTPOnly, Secure, SameSite=Strict)
- **Step 6:** Return success response with user role

Password Security:

- **Hashing Algorithm:** bcrypt with cost factor 12
- **Password Policy:** Minimum 12 characters, uppercase, lowercase, number, special character
- **Account Lockout:** 5 failed attempts = 30 minute lockout
- **Password Storage:** Never stored in plaintext; only bcrypt hash in database

8.2.2 Authorization (RBAC) Implementation

Role-Based Access Control with three roles:

Role	Permissions	Enforcement

Staff	View inventory, Update quantities	Middleware: RequireRole('staff')
Manager	Staff + Add products, Edit products	Middleware: RequireRole('manager')
Admin	Manager + User management, Full access	Middleware: RequireRole('admin')

Authorization Middleware Implementation:

- Every protected API route uses authorization middleware
- Middleware checks session validity and extracts user role
- Role verification occurs server-side (never trust client)
- Deny-by-default approach: no access unless explicitly granted
- Failed authorization returns 403 Forbidden with generic message

8.3 Data Protection Architecture

8.3.1 Encryption in Transit

- **Protocol:** HTTPS/TLS 1.2 or higher for all communications
- **Certificate:** Valid SSL/TLS certificate from trusted CA
- **Cipher Suites:** Strong ciphers only (AES-256-GCM, ChaCha20-Poly1305)
- **HSTS:** HTTP Strict Transport Security header enforces HTTPS

8.3.2 Encryption at Rest

- **Database:** PostgreSQL transparent data encryption for sensitive tables
- **Backups:** Encrypted using AES-256 before storage
- **Logs:** Sensitive data redacted; encrypted when archived

8.3.3 Sensitive Data Handling

- **Passwords:** Never logged or transmitted; bcrypt hashed only
- **Session Tokens:** HTTPOnly cookies prevent JavaScript access
- **Database Credentials:** Stored in environment variables, not in code
- **API Keys:** Secrets management system with rotation policy

8.4 Security Controls and Defense in Depth

8.4.1 Rate Limiting

- **Implementation:** Fiber rate limiting middleware
- **Global Limit:** 100 requests per minute per IP address
- **Login Endpoint:** 5 requests per minute per IP (prevents brute force)
- **Response:** 429 Too Many Requests with Retry-After header

8.4.2 CSRF Protection

- **Token Generation:** Cryptographically random CSRF tokens per session
- **Token Delivery:** Embedded in forms and AJAX requests
- **Validation:** Server validates token on all state-changing operations (POST, PUT, DELETE)
- **SameSite Cookies:** Additional protection layer against CSRF

8.4.3 Security Headers

- Content-Security-Policy: Restricts script sources to prevent XSS
- X-Frame-Options: DENY (prevents clickjacking)
- X-Content-Type-Options: nosniff (prevents MIME type confusion)
- Strict-Transport-Security: Forces HTTPS for 1 year
- X-XSS-Protection: 1; mode=block (legacy browser protection)

8.4.4 Session Management

- **Session Timeout:** 30 minutes of inactivity
- **Session Storage:** Server-side in PostgreSQL sessions table
- **Session Regeneration:** New session ID after login (prevents session fixation)
- **Logout:** Destroys server-side session and clears cookie
- **Concurrent Sessions:** Allowed with audit logging

9. Security Components

This section details the modular security components that implement the security architecture defined in Section 3.

9.1 Input Validation Module

9.1.1 Validator Component

Centralized validation using Go validator package:

Location: pkg/validation/validator.go

Purpose: Whitelist validation for all user inputs

Key Functions:

- ValidateProductName(name string) error – Regex [A-Za-z0-9 .-] max 100
- ValidateQuantity(qty int) error – Range 0–999999
- ValidateEmail(email string) error – RFC 5322 format
- ValidatePassword(password string) error – Policy enforcement
- SanitizeInput(input string) string – HTML/script tag removal

9.1.2 SQL Injection Prevention

Implementation: Database access layer using prepared statements exclusively

Location: pkg/database/repository.go

Pattern:

db.Query("SELECT * FROM products WHERE id = $1", productID)

9.4 Encryption and Data Protection

9.4.1 TLS Configuration

Location: config/tls.go

Min Version: TLS 1.2

Cipher Suites:

- TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
- TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305

Certificate: Let's Encrypt or CA-signed certificate

9.4.2 Secrets Management

Storage: Environment variables loaded from .env (not in version control)

Access: config package provides type-safe access

Protected Secrets:

- DATABASE_URL – PostgreSQL connection string
- SESSION_SECRET – Session encryption key
- CSRF_SECRET – CSRF token generation secret

9.5 Security Middleware Stack

Fiber middleware applied in order:

1. Logger: Request/response logging
2. Recover: Panic recovery to prevent crashes
3. CORS: Cross-origin request handling
4. Security Headers: CSP, X-Frame-Options, etc.
5. Rate Limiter: Request throttling per IP
6. CSRF: Cross-site request forgery protection

7. Auth: Session validation (route-specific)
8. RBAC: Role-based authorization (route-specific)

10. Logging Design

Comprehensive logging for security monitoring, compliance, debugging, and incident response.

10.1 Logging Architecture

10.1.1 Log Levels

- **DEBUG:** Detailed information for development (disabled in production)
- **INFO:** General informational messages (API requests, successful operations)
- **WARN:** Warning messages (rate limit approached, unusual activity)
- **ERROR:** Error conditions (failed DB queries, validation failures)

- **SECURITY:** Custom level for security events (authentication failures, authorization denials)

10.1.2 Log Storage

- **Application Logs:** File-based (logs/app.log) with daily rotation
- **Security Logs:** Separate file (logs/security.log) with extended retention
- **Audit Logs:** PostgreSQL database (movement_history, auth_events tables)
- **Retention:** Application logs 30 days, Security logs 90 days, Audit logs 1 year
- **Format:** Structured JSON for machine parsing

10.2 User Action Logging

All inventory-affecting actions are logged with full context:

Log Entry Format:

{"timestamp":"2025-11-08T10:30:00Z","level":"INFO","user_id":42,"action":"UPDATE_QUANTITY","product_id":15,"quantity_before":100,"quantity_after":150,"change_type":"IN","ip_address":"192.168.1.100","user_agent":"Mozilla/5.0..."}

10.3 Security Event Logging

Critical security events logged with high priority:

Authentication Events:

- Login Success – User ID, IP, timestamp, user agent
- Login Failure – Username (if valid), IP, timestamp, reason

- Account Lockout – Username, IP, timestamp, attempt count
- Logout – User ID, session duration
- Session Timeout – User ID, last activity time

Authorization Events:

- Permission Denied – User ID, requested resource, required role, user role
- Privilege Escalation Attempt – User ID, suspicious behavior indicators
- Invalid Session Token – Session ID, IP address

Suspicious Activity:

- Rate Limit Exceeded – IP address, endpoint, request count
- SQL Injection Attempt – Detected pattern, IP, sanitized input
- XSS Attempt – Detected script, IP, affected field
- CSRF Validation Failure – Session ID, referrer, IP
- Unusual Access Pattern – User ID, behavior anomaly description

10.4 System Error and Exception Logging

Technical errors logged for debugging and system health monitoring:

- **Database Errors:** Connection failures, query errors, transaction rollbacks
- **API Errors:** 4xx/5xx responses with request context, stack traces (server-side only)
- **Validation Errors:** Failed input validation with field and reason
- **System Exceptions:** Panics, unexpected errors with full stack trace
- **Performance Issues:** Slow queries (>1s), high memory usage, timeout warnings

Error Response to User:

- Generic user-facing messages (no technical details)
- Detailed error information logged server-side only

- Correlation ID provided to user for support reference

10.5 Administrative Action Logging

All administrative actions require enhanced logging:

- **User Management:** Create, modify, delete user accounts
- **Role Changes:** Modifications to user roles or permissions
- **Configuration Changes:** System settings modifications
- **Audit Log Access:** Who viewed audit logs and when
- **Backup Operations:** Database backups initiated or restored
- **Data Export:** Large data exports or reports generated

Admin Log Table Schema:

Column	Description
admin_action_id	Primary key
admin_user_id	Foreign key to users
action_type	String
target_user_id	Nullable, for user management actions
details	JSON blob with action-specific data
ip_address	String
timestamp	Timestamp with time zone

10.6 Log Monitoring and Alerting

Automated monitoring for security incidents:

- **Real-time Alerts:** Multiple failed logins from same IP, privilege escalation attempts, SQL injection detection
- **Daily Digest:** Summary of security events, unusual activity patterns
- **Log Analysis Tools:** Scripts for parsing and analyzing JSON logs
- **Integration:** Future SIEM integration capability (syslog forwarding)

References

1. IEEE Software Requirements Specification (See

https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=720574)

2. IEEE Software Design Descriptions (See

https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=741934)